GECG10069 (561084) F26: Introduction to Programming (C++)

Lab 4 : Mathematical Operations



What you will learn from Lab 4

In this laboratory, you will practice mathematical operations and learn to use common mathematical functions in C++.

TASK 4-1 : COMPREHENSIVE EXAMPLE

✓

```
#include <iostream>
#include <cmath>
#include <cstdlib>
#include <ctime>
using namespace std;

#define PI 3.1415926

int main() {
    int a = 5, b = 2;
    cout << "a / b = " << a / b << endl;

    double x = 5.5;
    // cout << "x % 2 = " << x % 2 << endl;

    int angle = 90;
    cout << "sin(90) = " << sin(angle) << endl;

    // 沒有 srand()，rand() 每次結果一樣
    cout << "Random number (1~100): " << rand() % 100 << endl;

    double y = 2.5;
    cout << "floor(2.5) = " << floor(y) << endl;
    cout << "ceil(2.5) = " << ceil(y) << endl;
    cout << "round(2.5) = " << round(y) << endl;

    return 0;
}
```

EXERCISE 4-1: ACTIVATION FUNCTION

- ✓ Activation functions transform an input value into an output value and are commonly used in neural networks to introduce non-linearity, enabling neural networks to learn complex features and relationships from input data. A detailed discussion of activation functions may be introduced in Artificial Intelligence, Machine Learning or Deep Learning courses.
- ✓ You're asked to write a C++ program that computes three of commonly used activation functions in artificial neural networks: ReLU, Sigmoid and Softsign.
- ✓ For a given input x, calculate the output of following three activation functions:
  - ReLU:

$$ReLU(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

  - Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

  - Softsign:

$$Softsign(x) = \frac{x}{1 + |x|}$$

Input

- ✓ The input consists of one **floating-point** number *x* within the range of **-10.0 to 10.0**.
- ✓ Use *x* as the input to the three activation functions: ReLU, Sigmoid, and Softsign.

Output

- ✓ Output the values of the three activation functions: ReLU, Sigmoid, and Softsign.
- ✓ Display each value with exactly **4 digits** after the decimal point.
- ✓ Output format:

  ReLU: <value>

  Sigmoid: <value>

  Softsign: <value>

Sample Input 1
-2.0
Sample Output 1

Output:

```
ReLU: 0.0000
Sigmoid: 0.1192
Softsign: -0.6667
```

Sample Input 2

2.0

Sample Output 2

Output:

```
ReLU: 2.0000
Sigmoid: 0.8808
Softsign: 0.6667
```

Sample Code

```
#include <iostream>
#include <cmath>
#include <algorithm>
#include <iomanip>

using namespace std;

int main() {

  // define and read in the input from terminal
  double x;
  cin >> x;

  // calculate the outputs of each activation function
  double relu, sigmoid, softsign;

  // format output and display

  return 0;
}
```

<Hint>

- ✓ Recall the functions that were introduced in today's lecture.
  - ➢ e.g. `abs()`, `fabs()`, `max()`, `min()`……
- ✓ To calculate exponential, you may call `exp(x)` in `<cmath>`
- ✓ You may format output with `fixed` and `setprecision()` in `<iomanip>`
- ✓ 0.0 is a `double`, and 0.0f is a `float`.

EXERCISE 4-2: STELLAR MAGNITUDE AND BRIGHTNESS

- ✓ Tomori is a member of the astronomy club wondering how astronomers describe and compare the brightness of stars. In this exercise, you need to write a C++ program to help Tomori calculate the apparent magnitude difference between two stars from their observed fluxes.
- ✓ In astronomy, the difference in apparent magnitude between two stars is related to their observed fluxes by:

$$m_1 - m_2 = -2.5 \log_{10}\left(\frac{F_1}{F_2}\right)$$

- ✓ where m_1 and m_2 are the apparent magnitudes of the two stars, and F_1 and F_2 are their observed fluxes, respectively. A smaller apparent magnitude indicates a brighter object.

Input

- ✓ The input consists of two **positive floating-point** numbers F_1 and F_2, representing the observed fluxes of Star 1 and Star 2, respectively.
- ✓ F_1 and F_2 are within the range of **0.001 to 1000000**.

Output

- ✓ Output the apparent magnitude difference $m_1 - m_2$ between the two stars.
- ✓ Display the value with exactly **4 digits** after the decimal point.
- ✓ Output format:

  Magnitude Difference: <value>

Sample Input 1
1000 100
Sample Output 1
Output: `Magnitude Difference: -2.5000`
Sample Input 2
100 1000
Sample Output 2
Output: `Magnitude Difference: 2.5000`

Sample Code

```
#include <iostream>
#include <cmath>
#include <iomanip>

using namespace std;

int main() {

  // define variables and read the input data
  double f1, f2;
  cin >> f1 >> f2;

  // calculate the magnitude difference with log10() function

  // format output and display

  return 0;
}
```